

Dokumentacja projektu: Strona z kotkami i filmami „Kitty movies”

Spis treści

Cel projektu bazy danych.....	2
Opis dziedziny przedmiotowej.....	2
Założenia wstępne	2
Słownik pojęć	3
Wymagania	4
Instalacja	4
Opis głównych funkcji	5
1.Schemat bazy danych	5
2. Opis encji i relacji w bazie danych	7
3. Model MVC.....	9
4. Kontrolery	10
5. Użyte API.....	10
6. Struktura katalogów	11
Interfejs użytkownika.....	12
1. Opis GUI	12
2. Implementacja	13

Cel projektu bazy danych

Celem projektu jest stworzenie nowoczesnej i funkcjonalnej aplikacji webowej, która umożliwi użytkownikom kompleksowe zarządzanie kolekcją filmów. Aplikacja będzie korzystać zarówno z lokalnej bazy danych, jak i z zewnętrznego API The Movie Database (TMDB), co pozwoli na przeglądanie, wyszukiwanie oraz uzupełnianie informacji o filmach. Użytkownicy będą mogli dodawać własne filmy do lokalnej bazy danych, a także oznaczać wybrane tytuły jako ulubione, co pozwoli im na szybki dostęp do swojej indywidualnej kolekcji.

Dodatkowo, w celu zwiększenia atrakcyjności i elementu rozrywkowego aplikacji, projekt zakłada integrację z API Cataas, umożliwiającą codzienne wyświetlanie losowego obrazka przedstawiającego kota dnia.

Kolejnym istotnym celem projektu jest umożliwienie rejestracji i logowania użytkowników, a także zarządzanie ich danymi oraz przypisanymi preferencjami filmowymi. System będzie wspierał funkcje personalizacji i bezpiecznego dostępu do danych, umożliwiając tworzenie indywidualnych kont oraz ich administrację. Wszystkie dane będą przechowywane w wydajnej i spójnej bazie danych, która będzie podstawą do dalszej rozbudowy aplikacji.

Opis dziedziny przedmiotowej

Projektowana aplikacja jest skierowana do szerokiego grona użytkowników, w szczególności do miłośników kinematografii oraz entuzjastów kotów. Jej głównym założeniem jest stworzenie przyjaznej i intuicyjnej przestrzeni, w której użytkownicy mogą w prosty sposób uzyskiwać informacje o filmach – zarówno tych aktualnych, jak i starszych klasykach. Dzięki integracji z API TMDB możliwe będzie przeszukiwanie bogatej bazy danych filmów, zawierającej szczegółowe informacje takie jak opisy, gatunki, obsada, plakaty czy daty premier.

Aplikacja wyróżnia się także unikalnym elementem humorystyczno-rozrywkowym – codziennym wyświetlaniem losowego obrazka kota za pomocą API Cataas. Ma to na celu nie tylko zwiększenie przyjemności korzystania z aplikacji, ale także budowanie pozytywnego doświadczenia użytkownika poprzez wprowadzenie elementu zaskoczenia i uśmiechu.

Dzięki połączeniu funkcji informacyjnych i rozrywkowych aplikacja odpowiada na potrzeby współczesnych użytkowników internetu, którzy poszukują zarówno wartościowych treści, jak i lekkiej, przyjemnej formy interakcji. Projekt wpisuje się w rosnące zainteresowanie personalizowanymi rozwiązaniami webowymi, które łączą hobby z codzienną dawką pozytywnych emocji.

Założenia wstępne

- Wykorzystanie frameworka Laravel.
 - Integracja z API TMDB i Cataas.
 - Baza danych oparta na MySQL/MariaDB.
 - Wymagania techniczne: PHP 8.1+, Composer, Docker (opcjonalnie).
-

Słownik pojęć

- **Laravel** framework PHP wykorzystywany do tworzenia aplikacji webowych, oparty na architekturze MVC; zapewnia gotowe mechanizmy routingu, migracji bazy danych, uwierzytelniania i wiele innych.
 - **TMDB** – (The Movie Database) – zewnętrzne API umożliwiające pobieranie szczegółowych informacji o filmach, serialach, aktorach itp. Wymaga posiadania klucza API.
 - **Cataas** – darmowe API umożliwiające pobieranie losowych zdjęć kotów w formacie JSON, z możliwością filtrowania i dodawania podpisów.
 - **MVC** – Model-View-Controller, wzorzec architektoniczny stosowany w aplikacjach webowych.
 - **PHP 8.1+** – język programowania wykorzystywany po stronie serwera; wersja 8.1 wprowadza nowoczesne mechanizmy jak np. readonly properties, enumy czy funkcje first-class callable.
 - **Composer** – narzędzie do zarządzania zależnościami w projektach PHP; umożliwia instalację bibliotek (np. Laravel), ich aktualizację i autoloading klas.
 - **MySQL / MariaDB** – relacyjne systemy zarządzania bazami danych, używane do przechowywania danych aplikacji (np. użytkowników, filmów, kotów). MariaDB to darmowy fork MySQL zapewniający pełną kompatybilność.
 - **API** (Application Programming Interface) – interfejs pozwalający aplikacjom komunikować się ze sobą; np. aplikacja może pobierać dane z zewnętrznych serwisów (takich jak TMDB czy Cataas) przez API.
 - **Encja** – reprezentacja obiektu rzeczywistego w bazie danych, np. użytkownik (user), film (movie) czy obrazek kota (kotdnia); w Laravelu często odwzorowana jako model.
 - **Kontroler w Laravel** – klasa odpowiedzialna za logikę aplikacji; odbiera żądania HTTP, przetwarza dane (często korzystając z modeli), a następnie zwraca odpowiedź (np. widok lub dane JSON).
 - **Trasa (Route)** – definicja powiązania między adresem URL a odpowiednią funkcją w kontrolerze; Laravel używa systemu routingu do przekierowywania żądań do odpowiednich miejsc.
 - **GUI** (Graphical User Interface) – graficzny interfejs użytkownika, umożliwiający interakcję z aplikacją za pomocą elementów graficznych (np. przyciski, formularze, listy filmów).
 - **Endpoint** – konkretny adres URL w API, pod którym dostępna jest dana funkcjonalność lub zasób. Przykład:
`https://cataas.com/cat?type=medium&position=center&json=true` to endpoint API Cataas zwracający losowe zdjęcie kota w formacie JSON. Endpointy są wykorzystywane do komunikacji między aplikacjami i serwerami.
-

Wymagania

PHP 8.1+
Composer
Laravel 10+
MySQL/MariaDB
Klucz API TMDB (<https://www.themoviedb.org/>)
Dostęp do internetu (dla API kotów i TMDB)

Instalacja

- **Instalacja lokalna (bez Dockera)**

Sklonuj repozytorium:

git clone <https://github.com/StudiaCWinformatykaLKT/projektfilmy.git>

Zainstaluj zależności:

composer install

Skopiuj plik .env.example do .env i zaktualizuj ustawienia bazy danych oraz klucz TMDB:

cp .env.example .env

Wygeneruj klucz aplikacji:

php artisan key:generate

Uruchom migracje:

php artisan migrate

(Opcjonalnie) Uruchom seedery:

php artisan db:seed

Uruchom serwer deweloperski:

php artisan serve

Otwórz przeglądarkę i przejdź do <http://localhost:8000>.

Konfiguracja pliku .env

DB_CONNECTION=mysql

DB_HOST=localhost

DB_PORT=3306

DB_DATABASE=laravel

DB_USERNAME=root

DB_PASSWORD=

UWAGA

Aby aplikacja działała poprawnie na maszynie lokalnej, wymagane jest środowisko serwerowe. Jednym z popularnych rozwiązań jest XAMPP, który dostarcza niezbędne komponenty, takie jak Apache, PHP i MySQL.

Oficjalna strona XAMPP:

<https://www.apachefriends.org/pl/index.html>

- **Uruchamianie w Dockerze**

Upewnij się, że masz zainstalowane Docker i Docker Compose.

W głównym katalogu projektu powinny znajdować się pliki:

Dockerfile
docker-compose.yml
katalog docker/ z plikiem vhost.conf

Skonfiguruj plik .env

Ustaw w nim:

DB_CONNECTION=mysql
DB_HOST=db
DB_PORT=3306
DB_DATABASE=laravel
DB_USERNAME=laravel
DB_PASSWORD=laravel

Uruchom kontenery:

docker-compose up --build

Wejdź do kontenera aplikacji:

docker-compose exec app bash

Zainstaluj zależności i uruchom migracje:

composer install
php artisan key:generate
php artisan migrate

Aplikacja będzie dostępna pod adresem:

<http://localhost:8000>

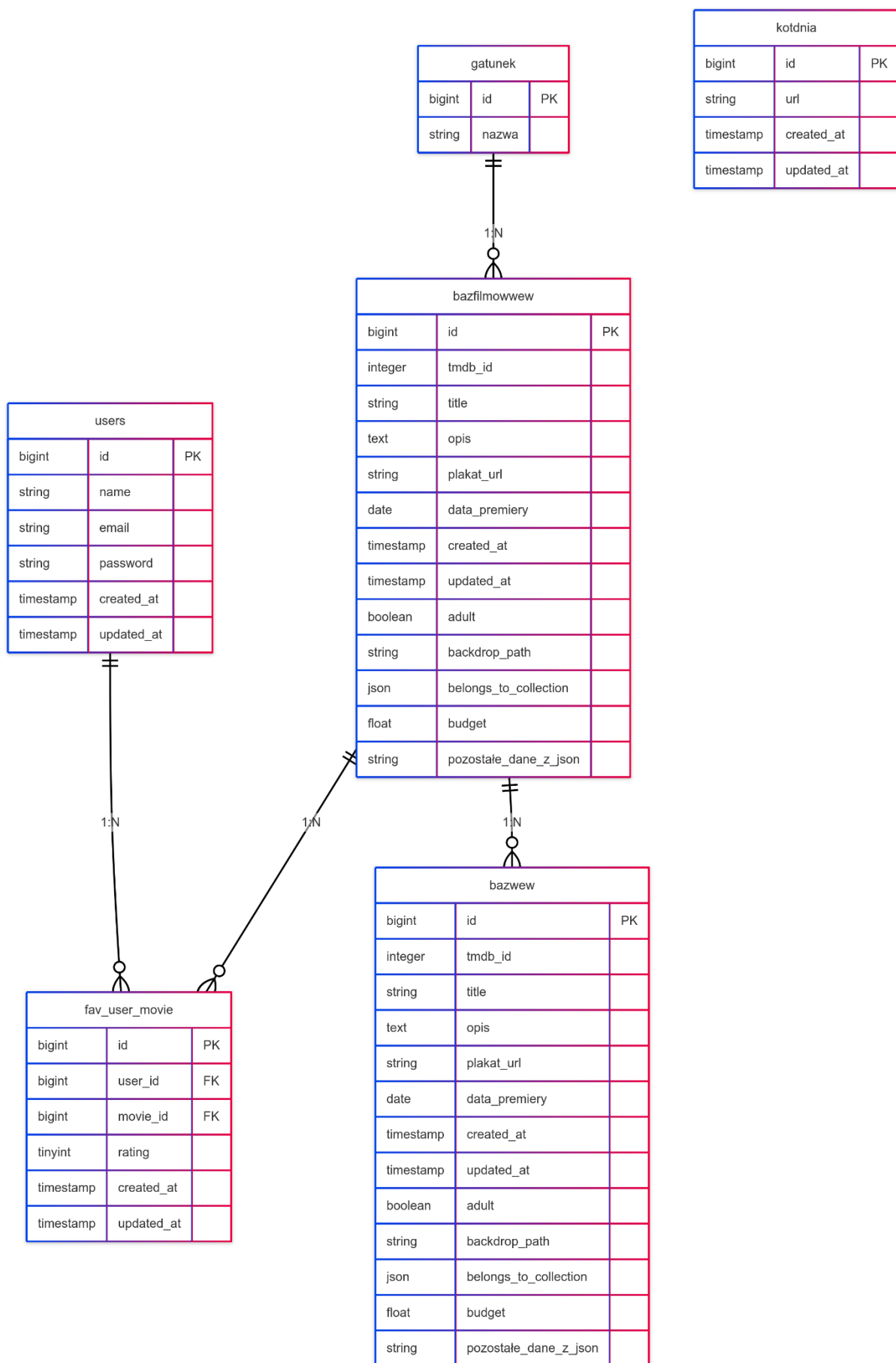
Użycie

- Wyszukiwanie filmów: Wprowadź tytuł filmu w polu wyszukiwania na górnym pasku nawigacyjnym i naciśnij przycisk wyszukiwania.
 - Zarządzanie użytkownikami: Przejdź do sekcji użytkowników, aby zobaczyć listę zarejestrowanych użytkowników.
-

Opis głównych funkcji

1.Schemat bazy danych

Diagram przedstawiający tabele users, gatunek, bazfilmowwew, kotdnia oraz relacje między nimi.



2. Opis encji i relacji w bazie danych

W projekcie zastosowano relacyjną bazę danych, która składa się z kilku głównych encji powiązanych relacjami typu jeden-do-wielu oraz wiele-do-wielu. Poniżej znajduje się szczegółowy opis poszczególnych tabel oraz ich wzajemnych zależności.

- **Encja users (Użytkownicy)**

Encja ta stanowi podstawową tabelę użytkowników systemu. Przechowuje informacje identyfikacyjne oraz uwierzytelniające (takie jak imię, e-mail i hasło). Każdy użytkownik może posiadać wiele filmów oznaczonych jako ulubione. Relacja ta realizowana jest poprzez tabelę pośredniczącą fav_user_movie, co tworzy związek typu wiele-do-wielu z encją bazfilmowwew.

- **Encja bazfilmowwew (Filmy – szczegółowa baza)**

Jest to główna tabela zawierająca kompletne dane dotyczące filmów, w tym tytuł, opis, język, popularność, datę premiery oraz gatunki. Informacje o gatunkach przechowywane są w postaci danych JSON w polach genres i genre_ids. Tabela ta uczestniczy w relacji wiele-do-wielu z encją users, dzięki tabeli fav_user_movie. Ponadto, relacja z encją gatunek jest reprezentowana pośrednio – poprzez wspomniane pola JSON.

- **Encja gatunek (Gatunki filmowe)**

Tabela gatunek przechowuje dane opisujące rodzaje gatunków filmowych, takie jak „komedia”, „dramat”, „akcja” itp. Jeden gatunek może być przypisany wielu filmom, co daje relację jeden-do-wielu względem tabel bazfilmowwew i bazfilmow. Relacja ta nie jest jednak realizowana bezpośrednim kluczem obcym, a poprzez wartości przechowywane w formacie JSON w filmach.

- **Encja fav_user_movie (Ulubione filmy użytkowników)**

Jest to tabela łącząca użytkowników z filmami oznaczonymi jako ulubione. Dodatkowo zawiera ona kolumnę rating, w której użytkownicy mogą przypisać ocenę wybranemu filmowi. Każdy rekord w tej tabeli odnosi się jednocześnie do jednego użytkownika oraz jednego filmu z tabeli bazfilmowwew, tworząc tym samym klasyczną relację pośredniczącą typu wiele-do-wielu.

- **Encja kotdnia (Kot dnia)**

Ta encja działa niezależnie od pozostałych i służy wyłącznie do przechowywania adresów URL losowych obrazków kotów pobieranych z zewnętrznego API Cataas. Każdy rekord zawiera datę dodania oraz link do obrazka. Tabela nie posiada żadnych relacji z pozostałymi encjami systemu, pełni natomiast funkcję estetyczną w interfejsie użytkownika.

- **Encja bazfilmow (Uproszczona baza filmów)**

Tabela ta stanowi serwerową wersję głównej bazy filmów (bazfilmowwew). Wykorzystywana do celów buforowania lub szybszego przetwarzania danych. W relacji z encją gatunek stosowane są pola JSON (genre_ids), a nie bezpośrednio klucze obce.

Podsumowanie kluczowych relacji:

- **Użytkownik ↔ Film** – relacja wiele-do-wielu, realizowana przez tabelę pośredniczącą fav_user_movie.
- **Film ↔ Gatunek** – relacja wiele-do-wielu reprezentowana poprzez dane JSON (genres, genre_ids), bez jawnych kluczy obcych.
- **bazfilmowwew ↔ bazfilmow** – relacja nieformalna, wskazująca na możliwość kopii lub buforowania danych między tymi tabelami.

Więzy integralności

Tabela fav_user_movie wykorzystuje więzy integralności w następujący sposób, aby zapewnić spójność danych między tabelami users i bazfilmowwew:

1. Klucz obcy do tabeli users:

```
$table->foreignId('user_id')->constrained('users')->onDelete('cascade');
```

W funkcji migracji bazy danych zastosowano klucz obcy user_id, który wiąże tabelę z danymi użytkowników. Kolumna ta, utworzona za pomocą metody foreignId('user_id'), przechowuje unikalne identyfikatory użytkowników, umożliwiając powiązanie ich z konkretnymi rekordami.

Metoda constrained('users') definiuje relację między tabelami, wymuszając, by każdy wpis w kolumnie user_id odpowiadał istniejącemu ID w tabeli users. Dzięki temu baza danych zachowuje spójność, unikając nieprawidłowych lub błędnych powiązań.

Dodatkowo, zastosowanie onDelete('cascade') zapewnia automatyczne czyszczenie powiązanych danych w przypadku usunięcia użytkownika. Gdy rekord z tabeli users zostanie skasowany, wszystkie związane z nim wpisy w tabeli fav_user_movie również znikną, eliminując ryzyko pozostawienia nieaktualnych lub zbędnych informacji.

Ta konfiguracja gwarantuje nie tylko poprawność relacji, ale też wygodę w zarządzaniu danymi, szczególnie w aplikacjach wymagających ścisłej zależności między użytkownikami a ich aktywnością.

2. Klucz obcy do tabeli bazfilmowwew:

```
$table->foreignId('movie_id')->constrained('bazfilmowwew')->onDelete('cascade');
```


W funkcji migracji bazy danych zastosowano klucz obcy `movie_id`, który wiąże tabelę z danymi filmów. Kolumna ta, utworzona za pomocą metody `foreignId('movie_id')`, przechowuje unikalne identyfikatory filmów, umożliwiając powiązanie ich z konkretnymi rekordami w systemie.

Metoda `constrained('bazfilmowwew')` definiuje relację między tabelami, wymuszając, by każdy wpis w kolumnie `movie_id` odpowiadał istniejącemu ID w tabeli `bazfilmowwew`. Dzięki temu baza danych zachowuje integralność, uniemożliwiając powiązanie z nieistniejącymi filmami.

Dodatkowo, zastosowanie `onDelete('cascade')` zapewnia automatyczne usuwanie powiązanych danych przy kasowaniu filmu. Gdy rekord z tabeli `bazfilmowwew` zostanie usunięty, wszystkie związane z nim polubienia i oceny w tabeli `fav_user_movie` również zostaną usunięte. Zapobiega to powstawaniu osieroconych wpisów i utrzymuje spójność bazy danych.

Ta konfiguracja zapewnia prawidłowe funkcjonowanie relacji między filmami a ich ocenami, jednocześnie upraszczając zarządzanie danymi w aplikacji.

3. Unikalność kombinacji `user_id` i `movie_id`:

```
$table->unique(['user_id', 'movie_id']);
```

Ten więz integralności zapewnia, że każda para `user_id` i `movie_id` musi być unikalna. Oznacza to, że jeden użytkownik może polubić/ocenić dany film tylko raz. Próba wstawienia zduplikowanej pary spowoduje błąd bazy danych.

Dzięki tym mechanizmom tabela `fav_user_movie` utrzymuje spójność referencyjną z tabelami `users` i `bazfilmowwew`, zapobiegając osieroconym rekordom i duplikatom preferencji.

3. Model MVC

Projekt oparty jest na architekturze MVC (Model-View-Controller), która pozwala na przejrzyste oddzielenie logiki biznesowej, prezentacji oraz obsługi danych. Struktura ta zwiększa czytelność kodu i ułatwia jego rozwój oraz utrzymanie:

- **Model** – odpowiada za warstwę danych aplikacji. Modele takie jak `User`, `Movie` czy `Gatunek` odwzorowują tabele w bazie danych i zawierają metody potrzebne do komunikacji z nimi, w tym relacje między encjami. Dzięki Eloquent ORM w Laravelu, modele są łatwe w obsłudze i zapewniają intuicyjny interfejs do pracy z danymi.
- **Widok (View)** – odpowiada za warstwę prezentacji. W projekcie wykorzystywane są pliki Blade (silnik szablonów Laravel), takie jak `films.blade.php`, `cat.blade.php` czy `dashboard.blade.php`, które generują dynamiczne strony HTML prezentujące dane użytkownikowi. Widoki mogą korzystać z przekazywanych z kontrolerów danych, aby wyświetlać listy filmów, szczegóły, obrazki kotów czy komunikaty.
- **Kontroler (Controller)** – stanowi pośrednika między modelem a widokiem i obsługuje logikę aplikacji. Przykładowo, `MainController` może odpowiadać za ładowanie strony głównej i pobieranie obrazka kota dnia, `MovieController` za

obsługę przeglądania i wyszukiwania filmów, a UserController za rejestrację i zarządzanie kontami użytkowników. Kontrolery decydują, jakie dane zostaną pobrane i które widoki mają zostać wyrenderowane.

4. Kontrolery

- **MainController**

cat() – wyświetla kota dnia

films() – wyświetla filmy z lokalnej bazy i z tabeli bazfilmowwew

user() – wyświetla użytkowników

search(Request \$request) – wyszukuje filmy przez API TMDB i lokalnie

getCatImage() – zwraca widok z losowym kotem

getCatImageUrl() – pobiera lub generuje URL kota dnia (z API cataas.com)

- **MovieController**

search(Request \$request) – wyszukuje filmy lokalnie i przez TMDB

show(Request \$request, \$id) – wyświetla szczegóły filmu (lokalnie lub z TMDB)

addToLocal(Request \$request) – dodaje film do lokalnej bazy bazfilmowwew

5. Użyte API

W projekcie wykorzystano dwa zewnętrzne API, które odgrywają kluczową rolę w dostarczaniu dynamicznych danych – jedno służy do obsługi filmów, a drugie do wyświetlania losowych obrazków kotów. Oba interfejsy znacznie rozszerzają funkcjonalność aplikacji i wpływają na jej atrakcyjność dla użytkowników.

- **TMDB (The Movie Database)**

TMDB to jedno z najbardziej rozbudowanych i popularnych API filmowych, oferujące dostęp do szczegółowych informacji o milionach filmów, seriali, aktorach oraz ekipach filmowych. W projekcie TMDB API wykorzystywane jest do następujących celów:

- **Wyszukiwanie filmów** na podstawie tytułu lub frazy kluczowej, co umożliwia użytkownikom szybkie znajdowanie interesujących produkcji.
- **Pobieranie szczegółowych danych filmowych**, takich jak opis, oceny, popularność, daty premier, plakaty, lista gatunków czy produkcja.
- **Uzupełnianie lokalnej bazy danych** o filmy, dzięki czemu aplikacja może działać wydajniej i szybciej, nawet bez ciągłego dostępu do internetu.

Aby móc korzystać z API TMDB, konieczne jest posiadanie klucza API, który należy zarejestrować na stronie TMDB po założeniu konta. Klucz ten jest przechowywany w pliku środowiskowym .env jako zmienna TMDB_API_KEY, co zwiększa bezpieczeństwo i umożliwia łatwe zarządzanie danymi konfiguracyjnymi bez konieczności ich umieszczania w kodzie źródłowym.

- **Cataas (Cat as a Service)**

Cataas to lekkie i proste w użyciu API oferujące losowe obrazki kotów. Jest ono wykorzystywane w aplikacji do wyświetlania tzw. **kota dnia**, co wprowadza humorystyczny i relaksujący element do codziennego korzystania z aplikacji. Ten pomysł stanowi dodatkową wartość dodaną, przyciągając użytkowników nie tylko funkcjonalnością, ale i przyjemną estetyką.

Szczegóły użycia:

- **Endpoint:**
`https://cataas.com/cat?type=medium&position=center&json=true`
Endpoint ten zwraca losowy obrazek kota w formacie JSON, zawierający m.in. ścieżkę do pliku graficznego. Parametry typu `medium` i `position=center` określają format i pozycję kota na obrazku.
- Obrazek pobierany z API może być następnie zapisywany do bazy danych (kotdnia) wraz z datą dodania, co umożliwia wyświetlanie go użytkownikom każdego dnia oraz ewentualne archiwizowanie ulubionych kotów.

Oba API są łatwe do integracji i posiadają dokumentację, która pozwala na dalsze rozszerzanie funkcjonalności aplikacji w przyszłości – np. o system rekomendacji filmów, galerie kotów, filtrowanie po gatunkach itp.

6. Struktura katalogów

Projekt aplikacji oparty na Laravelu posiada uporządkowaną strukturę katalogów, co ułatwia zarządzanie kodem, jego rozbudowę oraz późniejsze utrzymanie. Kluczowe elementy struktury to:

- **app/Http/Controllers**

Zawiera kontrolery aplikacji, które odpowiadają za obsługę żądań użytkowników, logikę biznesową oraz komunikację z modelami i widokami.

Przykładowe kontrolery:

- `MainController.php` – obsługuje stronę główną aplikacji oraz pobieranie obrazka kota dnia z API Cataas.
- `MovieController.php` – odpowiada za logikę przeszukiwania filmów, pobierania szczegółów z TMDB oraz zarządzania lokalną bazą danych.

- **database/seeder/DatabaseSeeder.php**

Plik odpowiedzialny za wypełnienie bazy danych przykładowymi danymi, np. filmami, gatunkami czy użytkownikami. Seeder ten jest przydatny przy pierwszym uruchomieniu aplikacji lub w środowiskach testowych.

- **resources/views**

Katalog z plikami widoków Blade – szablonami HTML renderowanymi dynamicznie na podstawie danych przekazywanych z kontrolerów.

Przykładowe pliki:

- `search.blade.php` – widok szczegółowy listy filmów zgodnych z wpisaną frazą
- `cat.blade.php` – zawiera sekcję prezentującą kota dnia.
- `layouts/lay.blade.php` – główny layout strony, zawierający nagłówek, nawigację i sekcję główną aplikacji.

- **routes/web.php**

Plik definiujący trasy aplikacji webowej, przypisujące konkretne adresy URL do odpowiednich metod w kontrolerach. Umożliwia np. połączenie żądania do /films z metodą MovieController@index.

- **public/**

Katalog dostępny publicznie z poziomu przeglądarki. Znajdują się tutaj:

- pliki graficzne (np. ikony, logo),
 - arkusze stylów CSS,
 - skrypty JavaScript,
 - oraz inne zasoby statyczne niezbędne do poprawnego działania i wyglądu aplikacji.
-

Interfejs użytkownika

1. Opis GUI

- **Strona główna**

Po wejściu na stronę użytkownik widzi pole umożliwiające wyszukiwanie filmów po tytule. Obok prezentowana jest sekcja „Kot dnia”, w której codziennie wyświetlany jest losowo dobrany obrazek kota z API Cataas, co nadaje aplikacji unikalny, humorystyczny charakter.

- **Lista filmów**

Zawiera zarówno filmy zapisane w lokalnej bazie danych, jak i wyniki wyszukiwań pobrane dynamicznie z API TMDb. Lista przedstawia tytuły filmów wraz z podstawowymi informacjami, np. datą premiery, oceną, językiem czy gatunkiem.

- **Szczegóły filmu**

Po kliknięciu w konkretny film użytkownik zostaje przeniesiony na stronę z jego pełnym opisem. Widoczne są tam szczegółowe dane, takie jak fabuła, obsada, czas trwania, budżet, plakaty czy powiązane gatunki.

- **Panel użytkownika**

Dostępny po zalogowaniu. Umożliwia użytkownikowi przeglądanie i zarządzanie jego ulubionymi filmami, dodawanie ich do swojej listy, a także ewentualne usuwanie lub filtrowanie według własnych preferencji.

2. Implementacja

Fragment kodu kontrolera MovieController:

Poniższy kod umożliwia wyświetlanie szczegółowych informacji o filmie, niezależnie od tego, czy pochodzi on z lokalnej bazy danych, czy z zewnętrznego API TMDB. Podczas ładowania danych sprawdzane jest, czy dany film z API nie został już wcześniej zapisany lokalnie. System dodatkowo udostępnia użytkownikowi listę filmów oraz losowy obrazek kota dnia. Informacje te są przekazywane do widoku, gdzie dynamicznie generowane są elementy interfejsu – np. przycisk "Dodaj do bazy", jeśli filmu nie ma jeszcze lokalnie, oraz adnotacja o źródle danych (np. „Dane z TMDB API”).

```
public function show(Request $request, $id)
{
    $source = $request->query('source', 'local');
    if ($source === 'local') {
        $movie = DB::table('bazfilmowwew')->where('id', $id)->first();
        $movieExistsInLocalDb = true;
    } else {
        $apiKey = env('TMDB_API_KEY');
        $response =
Http::get("https://api.themoviedb.org/3/movie/{$id}", [
            'api_key' => $apiKey,
        ]);
        $movie = $response->json();
        $movieExistsInLocalDb = false;
        if (isset($movie['id'])) {
            $movieExistsInLocalDb = DB::table('bazfilmowwew')->where('tmdb_id', $movie['id'])->exists();
        }
    }
    $catImageUrl =
app(\App\Http\Controllers\MainController::class)->getCatImageUrl();
    $moviesWew = DB::table('bazfilmowwew')->get();
    return view('movie_show', compact('movie', 'source',
'moviesWew', 'catImageUrl', 'movieExistsInLocalDb'));
}
```

Kod odpowiadający za wyświetlenie "kota dnia" oraz trasa dla w kontrolerze

Tworzy trasę /cat, która przekierowuje zapytania HTTP typu GET do metody cat w kontrolerze MainController. Nadano jej nazwę cat, co ułatwia odwoływanie się do niej w kodzie (np. route('cat')).

```
Route::get('/cat', [MainController::class, 'cat'])->name('cat');
```

Funkcja ta odpowiada za pobranie i przechowanie obrazka "kota dnia". Najpierw sprawdza, czy w tabeli kotdnia istnieje już wpis przypisany do bieżącej daty. Jeśli taki wpis istnieje, funkcja zwraca zapisany wcześniej URL obrazka. W przeciwnym przypadku nawiązywane jest połączenie z API Cataas, z którego pobierany jest nowy obrazek kota. Następnie jego adres URL zostaje zapisany w bazie danych wraz z aktualną datą. Informacja o tym działaniu zostaje również zarejestrowana w logach aplikacji.

```
public function getCatImageUrl()
{
    $today = now()->toDateString();
    $catOfTheDay = DB::table('kotdnia')->whereDate('created_at',
    $today)->first();

    if ($catOfTheDay) {
        return $catOfTheDay->url;
    } else {
        $response =
Http::get('https://cataas.com/cat?type=medium&position=center&js
on=true');
        $data = $response->json();
        $newUrl = $data['url'];
        DB::table('kotdnia')->insert([
            'created_at' => $today,
            'url' => $newUrl,
        ]);
        Log::info('Nowy wpis dodany do bazy danych.');
```

```
        return $newUrl;
    }
}
```

Widok losowego wyniku wyszukiwania z Api

Ten fragment kodu przedstawia formularz wyszukiwania filmów, który znajduje się najprawdopodobniej w górnej części interfejsu (np. w pasku nawigacyjnym). Formularz wysyła żądanie typu GET do trasy movies.search, umożliwiając użytkownikowi wpisanie zapytania tekstowego w celu wyszukania filmu. Polu tekstowemu przypisano klasę stylów Bootstrap, aby było estetyczne i responsywne – posiada jasne tło oraz zaokrąglone krawędzie. Obok pola znajduje się przycisk z ikoną lupy, który po kliknięciu uruchamia wyszukiwanie. Cały formularz jest ukryty na bardzo małych ekranach (d-none d-sm-inline-block), co poprawia jego wygląd na urządzeniach mobilnych.

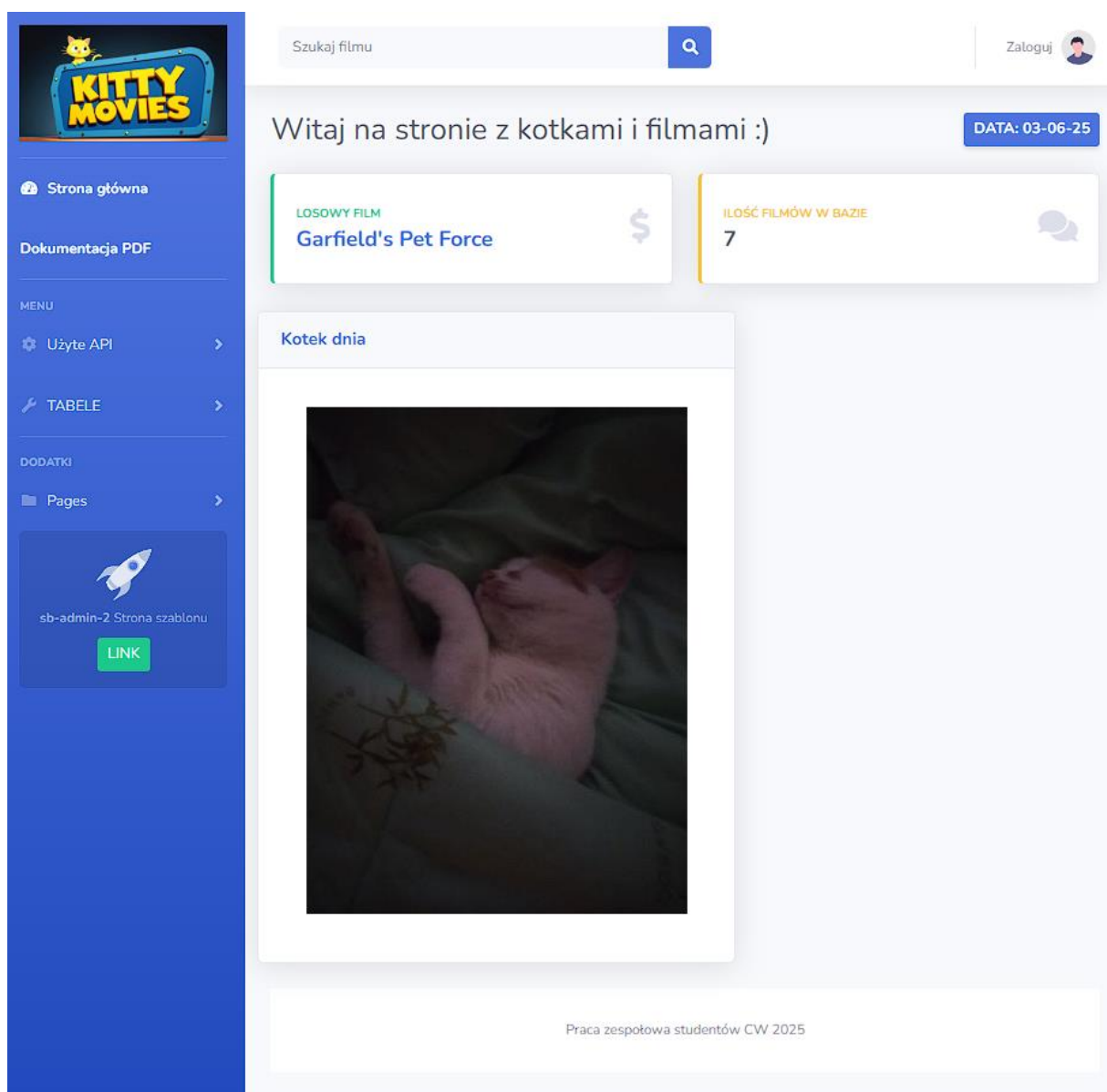
```
<form class="my-2 mr-auto d-none d-sm-inline-block form-inline
ml-md-3 my-md-0 mw-100 navbar-search"
```

```

action="{{ route('movies.search') }}" method="GET">
  <div class="input-group">
    <input type="text" class="border-0 form-control bg-
light small" name="query"
    placeholder="Szukaj filmu" aria-label="Search" aria-
describedby="basic-addon2">
    <div class="input-group-append">
    <button class="btn btn-primary" type="submit">
      <i class="fas fa-search fa-sm"></i>
    </button>
    </div>
  </div>
</form>

```

Główny format strony



Znalezienie filmów pasujących do wpisanej frazy star wars

Witaj na stronie z kotkami i filmami :)

Wyszukiwanie

- [Star Wars \(1977\)](#)
- [Star Wars: The Force Awakens \(2015\)](#)
- [Creatures from Star Wars \(2006\)](#)
- [Star Wars: The Rise of Skywalker \(2019\)](#)
- [Rogue One: A Star Wars Story \(2016\)](#)
- [Star Wars Kid: The Rise of the Digital Shadows \(2022\)](#)
- [Doraemon the Movie: Nobita's Little Star Wars 2021 \(2022\)](#)
- [Star Wars: The Last Jedi \(2017\)](#)
- [Star Wars: Episode I - The Phantom Menace \(1999\)](#)
- [Star Wars: Episode III - Revenge of the Sith \(2005\)](#)
- [LEGO Star Wars Terrifying Tales \(2021\)](#)
- [Doraemon: Nobita's Little Star Wars \(1985\)](#)
- [LEGO Star Wars Summer Vacation \(2022\)](#)
- [Solo: A Star Wars Story \(2018\)](#)
- [Star Wars_Skeleton_Crew \(2024\)](#)
- [Star Wars: Visions "Black" \(2025\)](#)
- [Star Wars: Episode II - Attack of the Clones \(2002\)](#)
- [Robot Chicken: Star Wars Episode III \(2010\)](#)
- [The Phantom Budget - A Star Wars Parody \(2025\)](#)
- [From Star Wars to Star Wars: The Story of Industrial Light & Magic \(1999\)](#)

Wybór konkretnego filmu

Szukaj filmu

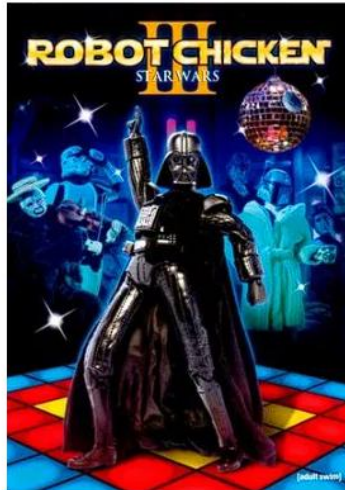


Witaj na stronie z kotkami i filmami :)

Robot Chicken: Star Wars Episode III

Rok premiery: 2010-12-19

Opis: Robot Chicken: Star Wars Episode III, directed by Chris McKay, combines the satirical with characters of the Star Wars universe.



Dodaj film do bazy